Architecture and Components Design Document (ACDD)

Student Attendance Management System (SAMS)

Version: 1.0

Prepared By:
Group ID: 12
Rishin Mallick (2330106)
Rishabh Rai (2330105)
Shashank Pitaliya (2330189)
Sunandan (2330198)

Submitted To:
D.K. Rout, KIIT University

March 1,2026

Project Title:

Student Attendance Management System (SAMS)

Document Title:

Architecture and Components Design Document

Version: 1.0

Date: March 1,2026

Author(s) / Contributors:

Group 12
Rishin Mallick (2330106)
Rishabh Rai (2330105)
Shashank Pitaliya (2330189)
Sunandan (2330198)

Project Sponsor / Team Lead:

D.K. Rout, KIIT University

Brief Project Overview / Purpose:

The Student Attendance Management System (SAMS) is a web-based system designed to digitize attendance tracking in educational institutions. The purpose of this document is to define the system architecture, component structure, and user interface design derived from the approved Software Requirements Specification (SRS).

Version History

Version	Date	Author(s)	Changes Made
1.0	March 1,2026	Group 12	Initial Draft

Table of Contents

1. **Project Context & Requirements Integration**
    - 1.1 Purpose of Architecture Document
    - 1.2 System Overview (from SRS)
    - 1.3 Key Functional Requirements (Architecturally Significant)
    - 1.4 Key Non-Functional Requirements (Architecturally Significant)
    - 1.5 Assumptions, Dependencies, Constraints
    - 1.6 Requirements Analysis Diagrams (Placeholders)

2. System Architecture Design
    - 2.1 High-Level Architecture View
    - 2.2 Component-Level Design
        - 2.2.1 Authentication & RBAC Module
        - 2.2.2 Attendance Management Module
        - 2.2.3 Admin Management Module
        - 2.2.4 Reporting Module
        - 2.2.5 Notification Module
        - 2.2.6 Audit Logging Module
    - 2.3 Data Design View (High-level)
    - 2.4 Deployment Architecture
    - 2.5 Key System Workflows (Sequence/Communication Diagrams)
    - 2.6 Refinement & Refactoring Notes

3. User Interface (UI) & Experience (UX) Design
    - 3.1 GUI Interaction Flow (Swimlane Diagram)
    - 3.2 UI Layout Prototypes / Mockups
        - 3.2.1 Login Screen
        - 3.2.2 Student Dashboard
        - 3.2.3 Teacher Attendance Marking Screen
        - 3.2.4 Admin Approval Screen
    - 3.3 Minimal UI Notes (Responsive + Accessibility)

Section 1 – Project Context & Requirements Integration

1.1 Purpose of this Section

This section highlights the requirements that directly influence the architecture of SAMS. The goal is to ensure the proposed design supports core functionality, meets security expectations, performs well under load, and remains scalable as usage grows.

1.2 System Overview (from SRS)

SAMS replaces the traditional manual attendance register with a centralized web platform. Teachers mark attendance for the classes they handle, students can view attendance (subject-wise and overall percentage), and administrators manage system data such as users, subjects, and classes. Administrators also approve or reject attendance correction requests.

1.3 Key Functional Requirements Summary (Architecturally Significant as per FR IDs in the SRS)

FR-01: The system supports user registration and secure login

FR-02: Teachers can mark daily attendance for assigned classes

FR-03: Students can view subject-wise attendance and overall percentage.

FR-04 + FR-05: Teachers raise correction requests; admins approve/reject them

FR-06: Attendance data is securely stored in the database.

FR-07: Admins manage users, classes, and subjects.

FR-08: Attendance reports can be generated for academic/administrative use.

FR-09: Notifications via email/SMS (optional, low priority)

1.4 Key Non-Functional Requirements Summary (Architecturally Significant)

Performance: Typical system actions should complete within 3 seconds, and the system should support at least 500 concurrent users

Security: Use HTTPS, enforce encryption where needed, store passwords securely, and implement RBAC

Reliability: System available during institutional hours, maintenance scheduled, and database backups enabled

Usability: Dashboards should be simple and role-specific, requiring minimal training

Compliance: The system must follow institutional attendance rules and basic data privacy requirements

Constraints: Internet connectivity is mandatory; notifications depend on third-party gateways; deployment requires admin verification of accounts

1.5 Assumptions, Dependencies, Constraints

Assumptions

Users are comfortable with basic computer and browser usage

Teachers enter attendance correctly

Server uptime is ensured during expected working hours

Dependencies

Email/SMS gateway services (only if notifications are enabled)

Database services remain available and stable

Constraints

All system communication must use HTTPS.

Sensitive operations must be role-restricted (RBAC).

Institutional attendance rules (minimum %, correction policies) must be enforced

1.6 Requirements Diagrams (Placeholders Only)

Figure 1-1: Use Case Diagram (Student/Teacher/Admin)

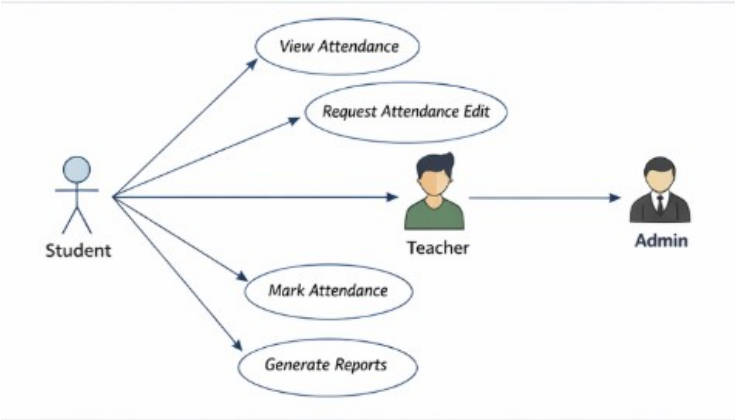


Figure 1-2: System Context Diagram

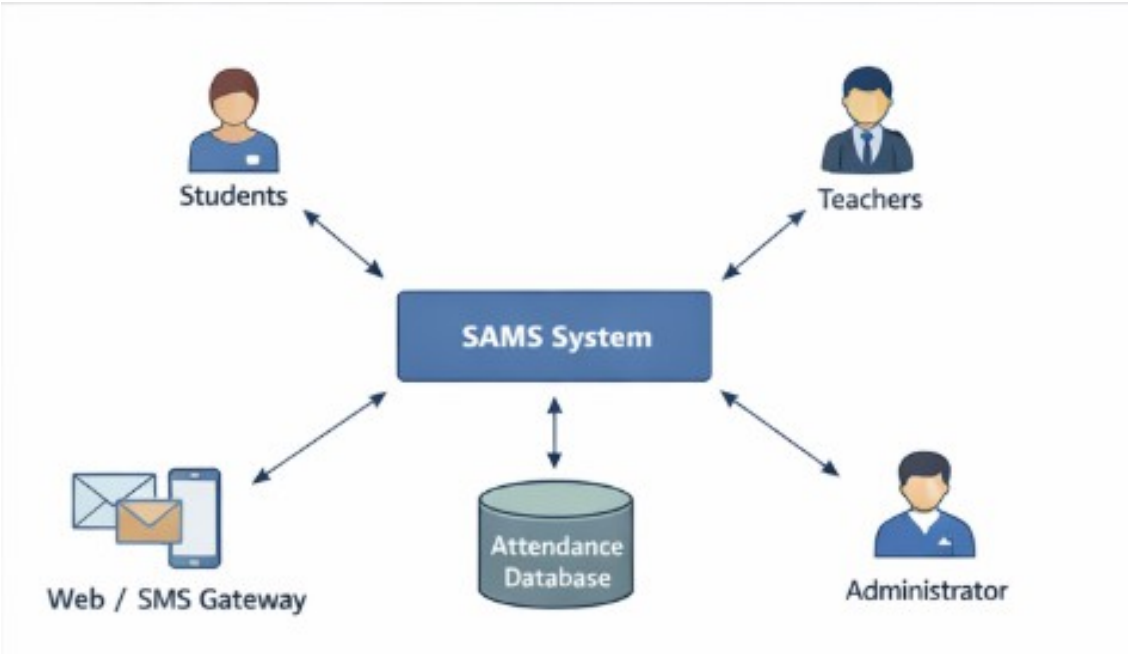


Figure 1-3: Actor Descriptions / User Roles Summary

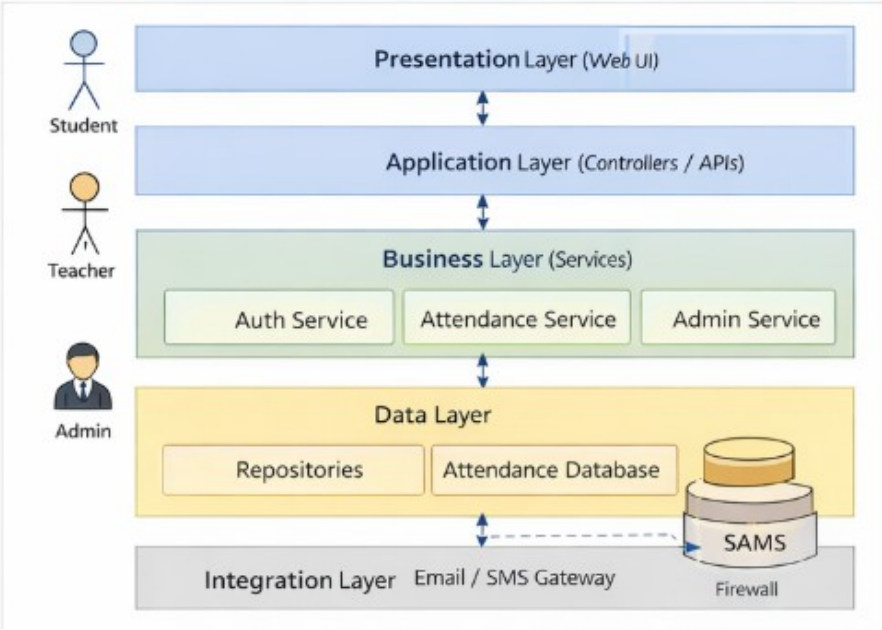
Actor	Description	Key Responsibilities
Student	Enrolled user accessing the system	• View attendance • *Request attendance corrections*
Teacher	*Instructor managing class attendance*	• Mark student attendance • *Generate attendance reports*
Admin	*System administrator / manager*	• *Manage users, classes, subjects* • *Approve or reject **attendance** edits*

Section 2 — System Architecture Design

2.1 High-Level Architecture View (UML Placeholder)

Goal: Show major subsystems, external services, and interactions.

Figure 2-1: Layered Architecture Diagram



Presentation Layer (Web UI)

This is the front-end layer where users interact with the system through a web browser. It provides role-based dashboards for students, teachers, and administrators. It handles form inputs (like login and attendance marking), displays attendance data, and shows notifications or reports.

Application Layer (Controllers / APIs)

This layer acts as the communication bridge between the UI and the business logic. It receives requests from the Presentation Layer, validates inputs, routes requests to the appropriate service, and returns responses. It exposes APIs such as login, mark attendance, view attendance, and manage users.

Business Layer (Services)

This layer contains the core system logic. It implements rules related to authentication, attendance processing, report generation, approval workflows, and role-based access control. Services such as Auth Service, Attendance Service, and Admin Service operate here.

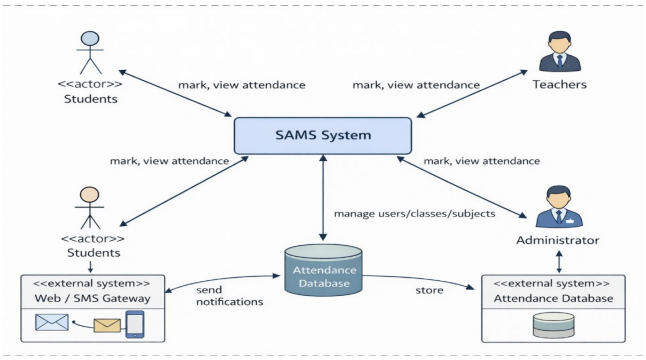
Data Layer (Repositories / Database)

This layer is responsible for storing and retrieving data. It includes repositories that interact with the Attendance Database. All persistent data such as users, roles, attendance records, subjects, and edit requests are managed here.

Integration Layer (Email / SMS Gateway)

This layer handles communication with external services. It connects SAMS to third-party email or SMS gateways to send attendance notifications, warnings, or alerts. It ensures external integrations remain separate from core business logic.

Figure 2-2: Context Diagram



2.2 Component-Level Design

2.2.1 Authentication & RBAC Module

Purpose : Handles login, session/token management, role-based access enforcement.

Interfaces (list):

1)POST /auth/login

2)POST /auth/register

3)GET /auth/logout

4)GET /auth/me

5)RBAC middleware/guard interface

UML diagrams :

Figure 2-3: Component Diagram — Auth & RBAC

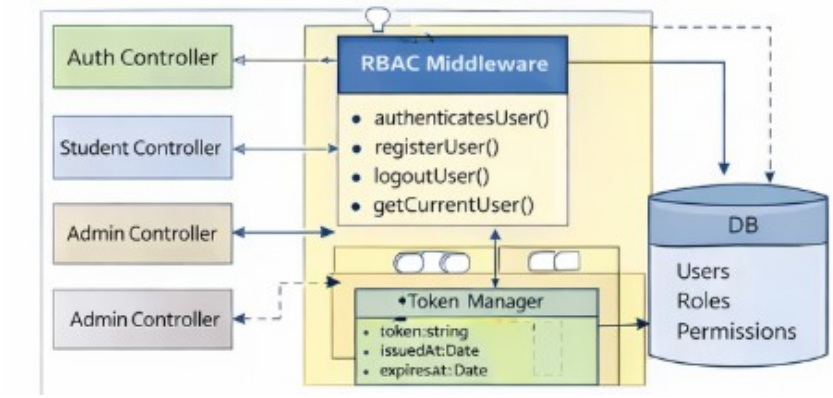
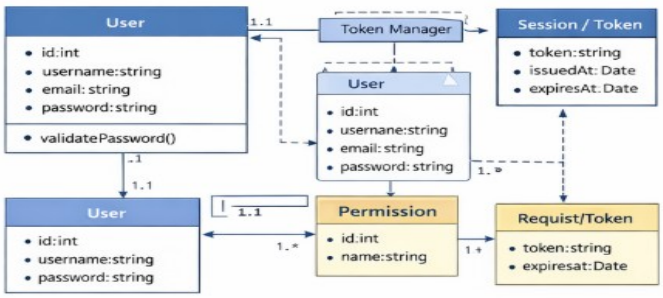


Figure 2-4: Class Diagram — User, Role, Permission, Session/Token



2.2.3 Admin Management Module

Purpose: Admin manages users/classes/subjects and approves edit requests.

Interfaces:

1)POST /admin/users

2)POST /admin/classes

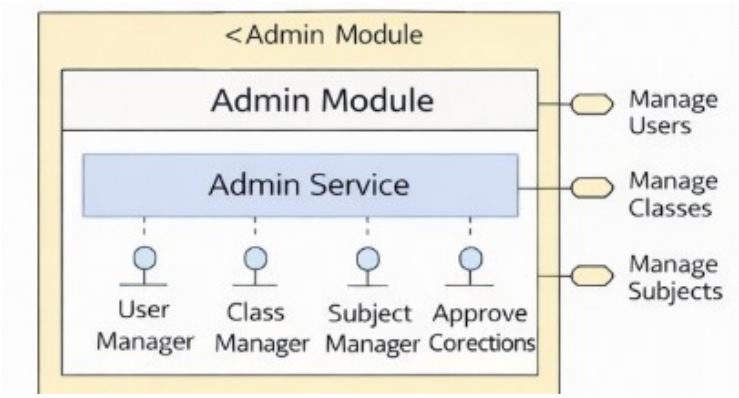
3)POST /admin/subjects

4)POST /admin/edit-request/{id}/approve

5)POST /admin/edit-request/{id}/reject

UML placeholders:

Figure 2-7: Component Diagram — Admin Module



2.2.4 Reporting Module

Purpose: Generate attendance reports for teachers/admin (view/download).

Interfaces:

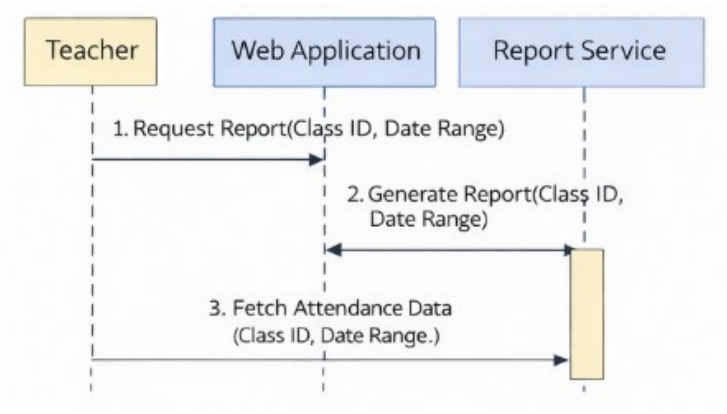
1)GET /reports/class/{id}

2)GET /reports/student/{id}

3)GET /reports/export (PDF/CSV) — [if applicable]

UML placeholders:

Figure 2-8: Sequence Diagram — Generate Report



2.2.5 Notification Module

Purpose: Email/SMS alerts for attendance updates / low attendance warnings.

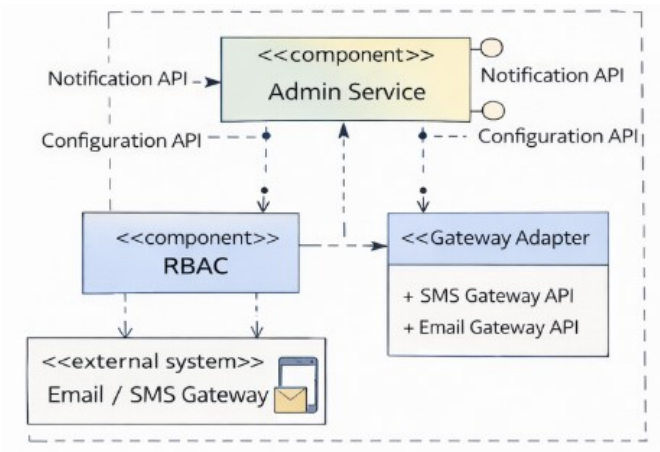
Interfaces:

1)POST /notify/attendance-warning

2)Integration adapter interface to gateway

UML placeholders:

Figure 2-9: Component Diagram — Notification + Gateway Adapter



2.2.6 Audit Logging Module

Purpose: Log critical actions (login attempts, attendance changes, approvals/rejections).

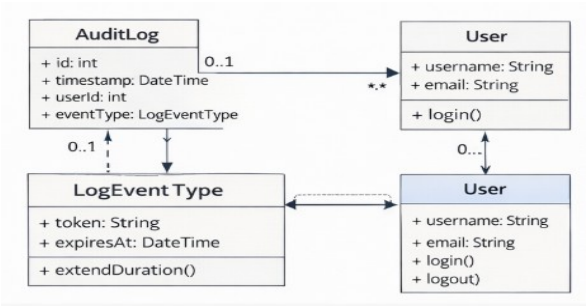
Interfaces:

1)Internal logging interface used by all modules

2)Admin log viewer endpoint (optional)

UML placeholders:

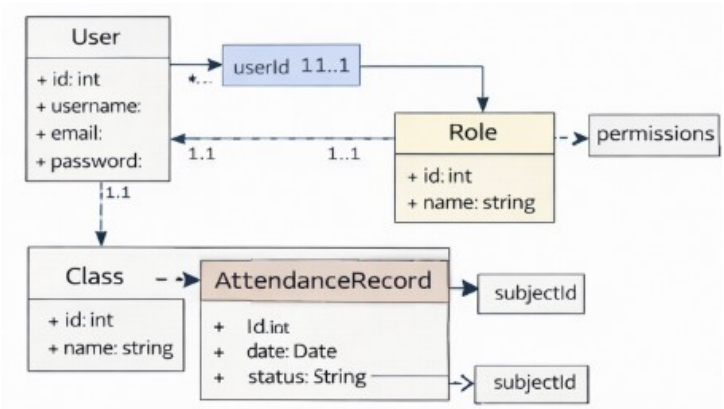
Figure 2-10: Class Diagram — AuditLog, LogEventType



2.3 Data Design View (High-Level)

Main Entities: User, Role, Class, Subject, AttendanceRecord, EditRequest, AuditLog

Figure 2-11: ER Diagram / UML Class Diagram (Data Model)



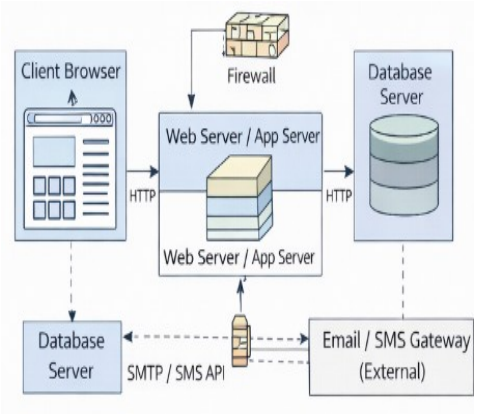
2.4 Deployment Architecture

Goal: Show where components run (client, server, DB, external services).

Figure 2-12: UML Deployment Diagram

Include nodes like:

- Client Browser (Student/Teacher/Admin)
- Web Server / App Server
- Database Server
- Email/SMS Gateway (External)



2.5 Key System Workflows (UML placeholders)

Figure 2-13: Sequence Diagram — Teacher Marks Attendance

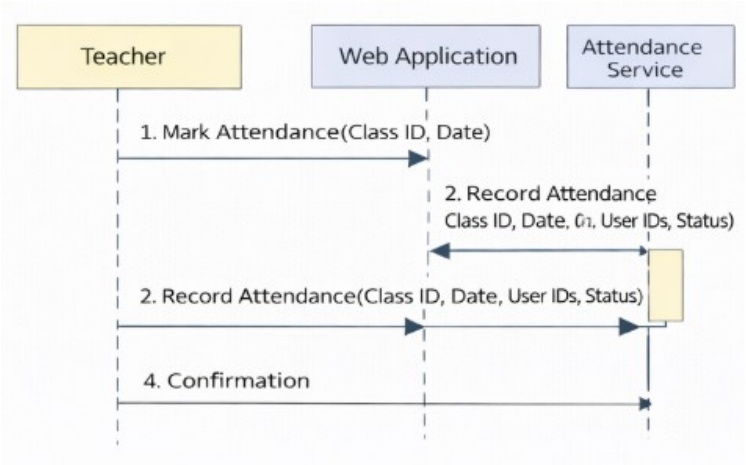


Figure 2-14: Sequence Diagram — Student Views Attendance

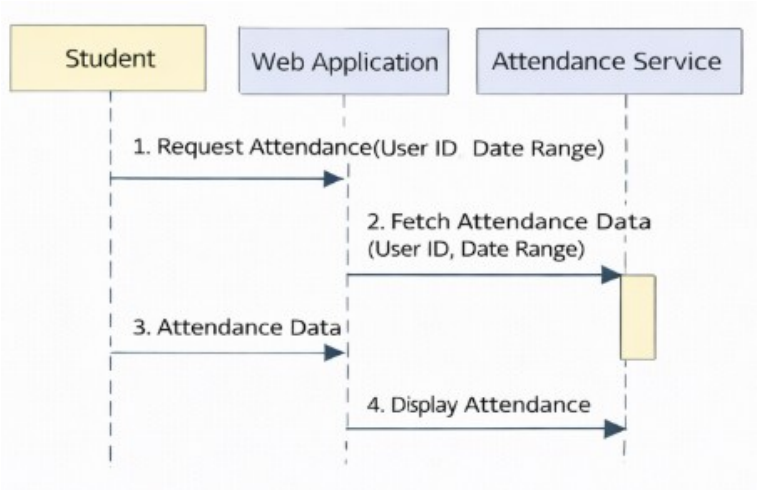
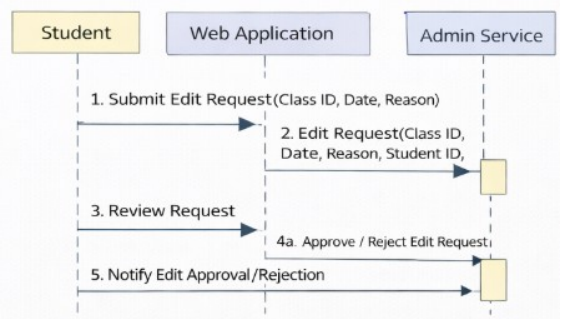


Figure 2-15: Sequence Diagram — Edit Request → Admin Approval



2.6 Refinement & Refactoring Notes

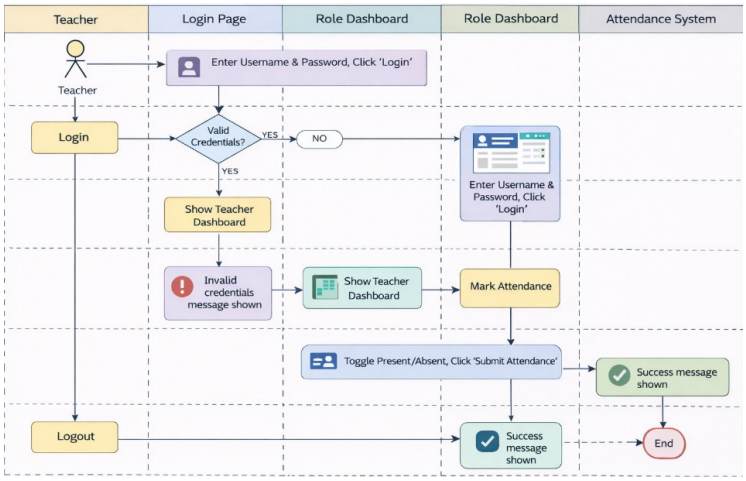
- Initial design: single attendance module handling marking + approval.
- Refined design: separated Edit Request Workflow into Admin module for control.
- Added Audit Logging as cross-cutting concern for traceability.
- Introduced gateway adapter to isolate third-party notification dependency.

Section 3 — UI & UX Design

3.1 GUI Interaction Flow (Swimlane Diagram Placeholder)

Login → role dashboard → action → system response → logout

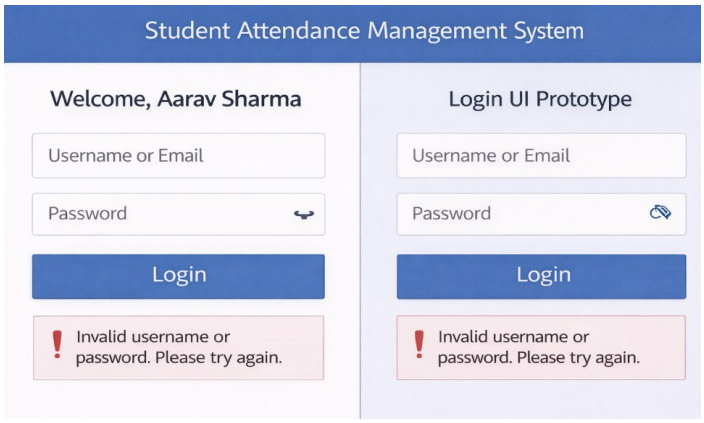
Figure 3-1: Swimlane Diagram — Teacher Marks Attendance Flow



3.2 UI Layout Prototypes / Mockups

3.2.1 Login Screen Mockup

Figure 3-2: Login UI Prototype



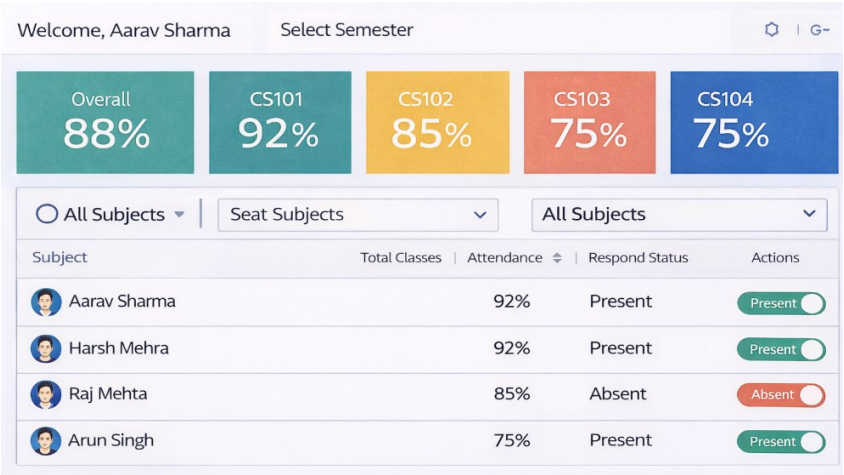
Minimal notes:

- **Username/Email Field:** Input field for registered username or email ID.
- **Password Field:** Secure input field with hidden characters.
- **Login Button (Prominent):** Clearly visible primary action button placed below input fields.

- **Error Message Area:** Displays validation errors (e.g., invalid credentials) below the login button.

3.2.2 Student Dashboard Mockup

Figure 3-3: Student Dashboard



Minimal notes:

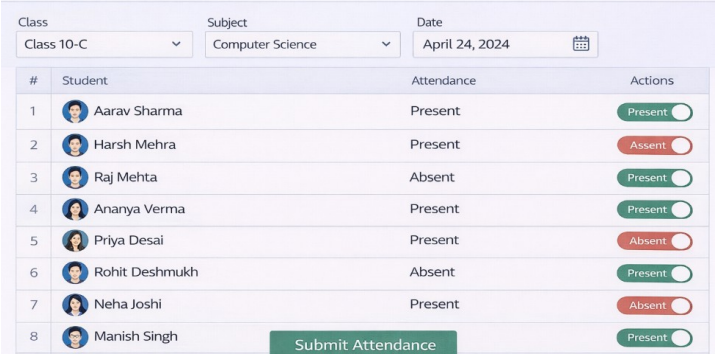
- **Attendance Summary Cards:** Display overall attendance percentage and subject-wise percentages at the top for quick overview.

- **Notification Panel:** Shows alerts such as low attendance warnings or important updates.

- **Filter by Subject/Semester:** Dropdown options to filter attendance records based on selected subject or semester.

3.2.3 Teacher Attendance Marking Screen Mockup

Figure 3-4: Teacher Attendance Screen



Minimal notes:

- **Select Class + Subject + Date:** Dropdown menus and date picker at the top to choose the specific class, subject, and attendance date.
- **Student List with Present/Absent Toggles:** Displays enrolled students with toggle switches to mark attendance status.
- **Submit Attendance Button:** Prominent action button to save and record the marked attendance.

3.2.4 Admin Approval Screen Mockup

Figure 3-5: Admin Edit Request Approval

Minimal notes:

- **List of Edit Requests with Status:** Displays submitted attendance correction requests along with relevant details and current status.

- **Approve/Reject Buttons:** Allows the administrator to either approve or reject the selected request.

- **View Request Details Modal/Panel:** Shows detailed information about the selected edit request before taking action.

3.3 Minimal UI Notes

- Responsive layout for mobile + desktop
- Clear validation messages for forms
- Role-specific navigation to prevent confusion
- Consistent button placement for submit/cancel actions
- Accessibility: readable font sizes, keyboard navigation for forms